UT2. MANEJO DE CONECTORES

ACTIVIDAD 1

Conexión con base de datos relacional

Título: Establecer conexión con base de datos.

Descripción:

En esta actividad, implementarás una clase Conector en Java para conectar una aplicación con una base de datos MySQL. La aplicación intentará establecer una conexión y mostrará mensajes en la consola para indicar si la conexión ha sido exitosa o si ha habido algún error. Además, el programa debe asegurarse de cerrar la conexión cuando finalice, independientemente de si la conexión fue exitosa o no.

Objetivos:

Desarrollar una aplicación en Java que permita establecer una conexión a una base de datos MySQL y gestionar posibles errores de conexión. Esta actividad te ayudará a comprender y practicar la conexión a bases de datos en aplicaciones Java utilizando JDBC (Java Database Connectivity).

Instrucciones:

1. **Implementar la clase Conector:**

   - Crea una clase llamada Conector.
   - Define en esta clase las variables necesarias para almacenar la URL de conexión, el nombre de usuario y la contraseña de la base de datos.

2. **Código para conectar a la base de datos:**

   - Dentro de la clase Conector, implementa el método main que intente conectarse a la base de datos que ya creaste, utilizando la URL, el nombre de usuario y la contraseña configurados.
   - Utiliza DriverManager.getConnection() para intentar establecer la conexión. Si la conexión es exitosa, muestra el mensaje: "Conexión a la base de datos [nombre_de_base_de_datos] correcta.".
   - En caso de error, muestra un mensaje de error en la consola indicando que la conexión falló y captura la excepción SQLException para mostrar los detalles.

3. **Cerrar la conexión:**

   - Asegúrate de que la conexión se cierre una vez que el programa haya terminado, independientemente de si fue exitosa o fallida.

- Utiliza un bloque finally para verificar si la conexión está abierta y, de ser así, ciérrala usando connection.close().
- Si el cierre de la conexión es exitoso, muestra el mensaje: "Conexión cerrada.".

4. **Prueba del programa:**

- Ejecuta el programa en tu entorno y verifica que:
- Muestra el mensaje de conexión exitosa si la conexión ha sido correcta.
- En caso de error, muestra el mensaje de conexión fallida y detalles de la excepción.
- Al finalizar, solo **si la conexión llegó a abrirse**, el mensaje "Conexión cerrada." se mostrará en la consola.

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Presenta la tarea, el código es correcto, pero no ha demostrado su funcionamiento mediante la captura de pantalla
7	Presenta la tarea, el código es correcto, demuestra su funcionamiento mediante la captura de pantalla, pero no describe los principales métodos incluidos en el código.
9	Se completa la tarea correctamente en todos sus apartados
10	Se implementa una funcionalidad extra o un detalle innovador que mejora el programa.

Entrega:

1. **Incluye aquí el código fuente**:

```
package org.example;

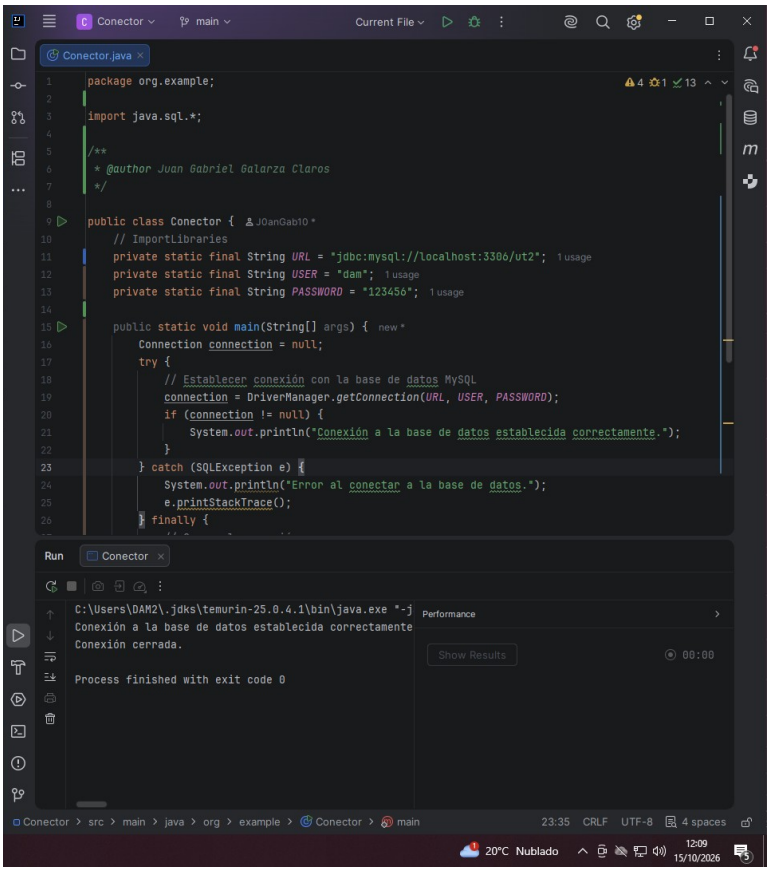
import java.sql.*;

/**
 * @author Juan Gabriel Galarza Claros
 */

public class Conector {
    // ImportLibraries
    private static final String URL = "jdbc:mysql://localhost:3306/ut2";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";
```

```
  public static void main(String[] args) {
      Connection connection = null;
      try {
          // Establecer conexión con la base de datos MySQL
          connection = DriverManager.getConnection(URL, USER, PASSWORD);
          if (connection != null) {
              System.out.println("Conexión a la base de datos establecida correctamente.");
          }
      } catch (SQLException e) {
          System.out.println("Error al conectar a la base de datos.");
          e.printStackTrace();
      } finally {
          // Cerrar la conexión
          if (connection != null) {
              try {
                  connection.close();
                  System.out.println("Conexión cerrada.");
              } catch (SQLException ex) {
                  ex.printStackTrace();
              }
          }
      }
  }
}
```

2. **<u>Incluye aquí una captura de pantalla que demuestre el correcto funcionamiento del programa desarrollado, y donde sea visible el autor</u>**:



i. **Describe aquí los métodos y variables principales utilizadas en el programa**:

- **Clases Principales**
  - Conector: Es la clase única del programa y contiene toda la estructura para ejecutar y gestionar la conexión con la base de datos MySQL.
- **Variables Principales**
  - URL: Guarda la dirección, el puerto (3306) y el nombre de la base de datos (ut2) a la que nos queremos conectar.
  - USER: Almacena el nombre de usuario.
  - PASSWORD: Contiene la contraseña de acceso.
  - connection: Objeto que sirve como canal de comunicación con la base de datos; empieza en null y se usa para comprobar el estado y cerrarlo al terminar.
- **Métodos Principales**
  - main: Es el único método y el punto de entrada del programa. Su funcionamiento sigue estos pasos:
    1. Intenta conectarse usando los datos de configuración mediante DriverManager.getConnection().
    2. Comprueba si la conexión se ha establecido correctamente y muestra un mensaje por pantalla.
    3. Captura posibles errores con el bloque catch para evitar que el programa falle de forma inesperada.
    4. Utiliza el bloque finally para asegurarse de que la conexión se cierre de forma segura (connection.close()) pase lo que pase.